

Lab 3: Layouts & Dynamic Routing

Part 1: Practical Tasks

Task 1: The Main Layout (The Application Skeleton)

- Create a MainLayout component.
- Inside this component, use your chosen UI library to build a fixed Navbar containing links to "Home" (/) and "Cart" (/cart).
- Below the Navbar, place the routing outlet component (<Outlet />) so that child pages will render inside this layout.

Task 2: Routing Configuration

- Set up your React Router configuration.
- Make the MainLayout the parent route.
- Configure the following child routes to render inside the layout:
 - / -> Renders the ProductsList component.
 - /product/:id -> Renders the ProductDetails component (Note the path parameter).
 - /cart -> Renders a simple Cart component.
 - * -> Renders a NotFound component (404 page).

Task 3: Products List & Query Parameters

- In the ProductsList screen render all ProductCard components
- **Implementing Query Parameters:** Add two or three "Filter" buttons at the top of the page (e.g., "Show Electronics", "Show Clothing").
- When a button is clicked, update the URL to include a query parameter (e.g., /?category=electronics).
- Read the query parameter from the URL. Display a dynamic heading at the top of the page that says, for example, "Currently Browsing: Electronics" (or "All Products" if no query parameter exists).

Task 4: Product Details & Path Parameters

- Ensure each ProductCard has a "View Details" button that links to /product/{id} (using the specific ID passed to that card).
- In the ProductDetails component, read the **Path Parameter** (id) from the URL.
- Render a styled screen that displays: "Fetching detailed data for Product ID: [Insert extracted ID here]". Add a button to navigate back to the home page.

[Bonus] Task 5: UI Library Mastery

- Use the layout grid system from your chosen UI library (like Material UI's `<Grid>` or Bootstrap's `<Row>/<Col>`) to make the `ProductsList` responsive: 1 item per row on mobile, 3 items per row on desktop screens.

Part 2: Technical Terminology (Review Before Coding)

1. Routing Mechanisms (Client-Side vs. Server-Side)

- **Server-Side Routing (Traditional Variant):** Every time a user clicks a link, the browser sends a request to the server, downloads a completely new HTML document, and refreshes the entire screen.
 - *Pros:* Excellent for initial page load speed and Search Engine Optimization (SEO).
 - *Cons:* Slower page-to-page navigation and visual "flashing" between clicks.
- **Client-Side Routing (React Router Variant):** JavaScript intercepts the link click. Instead of asking the server for a new page, it dynamically hides the current React components and renders new components instantly while updating the URL in the address bar.
 - *Pros:* Provides a seamless, lightning-fast user experience similar to a native desktop or mobile application (Single Page Application, or SPA).
 - *Cons:* Requires the initial JavaScript bundle to load before navigation works.

2. Main Layout (Shared Architecture)

- **Definition:** A structural design pattern where a parent component acts as a permanent wrapper or "skeleton" for the application. It contains UI elements that must remain continuously visible across all pages, such as a top Navigation Bar (Navbar) or a bottom Footer.
- **Execution Concept:** The Layout component renders these permanent elements and reserves an empty space in its structure. When the user navigates to different URLs, only the content inside that reserved space is swapped out.

3. The Outlet

- **Definition:** Outlet is a specific component provided by React Router. It acts as the exact placeholder or injection point within a Main Layout where the child route's components are rendered.
- **Architecture Analogy:** If the Main Layout is a picture frame, the `<Outlet />` is the empty glass in the middle where you slide in different photographs (the child pages).

4. Route Parameters (Path Parameters)

- **Definition:** A dynamic, variable segment built directly into the structure of a URL path, used to identify a specific, unique resource within the application architecture.
- **Syntax & Code Visibility:** In the route definition `/product/:id`, the colon (`:`) dictates that `id` is a variable, not a literal string. If a user visits `/product/105`, the application extracts `105` as the specific ID.

5. useParams Hook

- **Definition:** A special function (Hook) provided by React Router that allows a React component to read and extract Path Parameters directly from the current URL.
- **Data Tracing:** When a user visits `/product/105`, executing the `useParams` hook captures a JavaScript object containing `{ id: '105' }`, giving the component the exact data needed to know which product details to display.

6. Query Parameters (Search Parameters)

- **Definition:** Optional key-value pairs appended to the very end of a URL, starting with a question mark (`?`) and separated by an ampersand (`&`). They are utilized to sort, filter, or pass additional data to a page without altering the primary route path.
- **Syntax Tracing:** In the URL `http://localhost:3000/products?category=electronics&sort=asc`:
 - `?` designates the start of the query string.
 - `category` is a key, and `electronics` is its corresponding value.
 - `&` chains the next parameter.
 - `sort` is a key, and `asc` (ascending) is its corresponding value.

7. Conditional Rendering

- **Definition:** The process of instructing React to display entirely different UI elements or components depending on a specific boolean condition or state variable.
- **Execution Concept:** Evaluating a `stock` prop inside a product card. If `stock === 0`, React evaluates the condition as true and renders a red "Out of Stock" badge. If the condition is false, React renders a green "In Stock" badge.